

Navigation Destinations and List Composition

Public promotion decision and refined Phase 4-5 implementation contract

STATUS: PUBLIC VERTICAL SLICE APPROVED

Decision

The dashboard-private spike in Phases 0-2 is accepted as sufficient implementation evidence. Phase 4 (the public Remix navigation-list family) and Phase 5 (the Fortal recipe and dashboard migration) are approved for local implementation.

Gate disposition

- The second-real-consumer evidence gate is explicitly waived by product decision. It is waived, not passed.
- Every other boundary, behavior, semantics, host, platform, style, API, maintenance, and compact-mode gate remains binding.
- The public contract must remain a host-agnostic vertical list. Drawer, rail, sidebar, and panel concerns remain outside it.

Statements superseded by this amendment

- The cover status 'public API pending promotion gates' is replaced by 'public vertical slice approved.'
- The decision-record statement that the public API is not approved is replaced by this approval.
- The promotion and acceptance language requiring a second real application consumer is replaced by the explicit waiver above.

All original pages follow this amendment unchanged. Where the original text conflicts with these four amendment pages, this amendment controls.

Approved public v1 boundary

The public surface contains two small declarative data contracts and one medium, controlled component. Application composition remains the mechanism for shells and overlays.

Piece	Public v1 contract
Destination<T>	RemixNavigationDestination<T>: value, required visible label, optional icon, semanticLabel, enabled, focusNode, autofocus, and per-item ToggleStyler
Section<T>	RemixNavigationSection<T>: optional label plus a destination list with a documented non-mutation contract
List<T>	RemixNavigationList<T>: sections, nullable selectedValue, nullable onSelected, enabled, semanticLabel, excludeSemantics, style, styleSpec, and styleFrom
NavigationListSpec / Styler	container, section, sectionLabel, destinations, and destination style slots; destination reuses ToggleSpec
FortalNavigationList<T>	generated wrapper over the Fortal recipe in Phase 5

Customization ladder

- Data: value, label, icon, and section grouping.
- Interaction and accessibility: enabled, semanticLabel, focusNode, and autofocus.
- Visual defaults: NavigationListStyler at the list level.
- Per-item visual override: ToggleStyler merged after the default destination style.
- Resolved override: raw NavigationListSpec, authoritative over all fluent styles.
- Host composition: callers wrap the list in scrolling, SafeArea, sidebars, drawers, or other layout.

Explicitly deferred

Drawer or sheet lifecycle; panel width and borders; SafeArea and scrolling; headers and footers; routing, URLs, and history; nested groups; badges and trailing widgets; arbitrary row builders; compact icon-only mode and automatic tooltips; roving arrows and Home/End behavior.

A future RemixDrawer or RemixSheet requires a separate RFC. It must accept an arbitrary child and independently specify overlay ownership, scrim, drag and dismissal behavior, Escape, focus containment and restoration, and host capabilities. It must not depend on navigation destination data.

Phase 4 implementation contract

The generic type is constrained to `T extends Object` so null remains the no-selection sentinel. Selection is controlled exclusively by `selectedValue`.

```
RemixNavigationList<T extends Object>(  
  sections: List<RemixNavigationSection<T>>,  
  selectedValue: T?, onSelected: ValueChanged<T>?, enabled: bool,  
  semanticLabel: String?, excludeSemantics: bool,  
  style: NavigationListStyler, styleSpec: NavigationListSpec?,  
)
```

Selection and validation

- Every enabled activation invokes `onSelected` exactly once, including reselection.
- A null callback or disabled root disables every destination while preserving selected styling. Item enabled state combines with root state.
- Values are unique across all sections and provide stable internal `ValueKey` identity.
- A non-null `selectedValue` must match exactly one destination in debug builds. Release builds render an unmatched value as no selection.
- Labels and supplied semantic labels are nonblank after trimming. At most one destination requests autofocus.
- Empty lists are valid when selection is null. Empty sections and their orphan labels are skipped.

Semantics, keyboard, and host

- The root is a `SemanticsRole.navigation` landmark with explicit child nodes and an optional landmark label.
- Each destination has one outer button-and-selected semantic node. `RemixToggle` semantics are excluded; toggled state is never published.
- Section labels are separate header nodes and never merge into destination names.
- Tab and Shift+Tab follow visual order. Enter and Space activate through `RemixToggle` and `NakedToggle`.
- The list owns no scroll view and enforces vertical order in LTR and RTL.
- Ordinary use works under `WidgetsApp` without `Material`, `Overlay`, `Navigator`, `SafeArea`, or router.
- True URLs and documents remain `RemixLink` composition, not navigation destinations.

Style precedence

Without `styleSpec`, the root `NavigationListStyler` resolves the list anatomy and each destination `ToggleStyler` merges after the default destination style. With `styleSpec`, the raw resolved spec is authoritative and bypasses both fluent layers. Container, section, and destination-group directions are forced vertical after resolution; styling cannot reorder semantics.

Approved execution sequence

Phase 4 - Remix public vertical slice

- Add the destination and section contracts, `RemixNavigationList`, and `NavigationListSpec/Styler`.
- Export the family from the Remix package barrel and add reader-facing component documentation.
- Cover construction, generic `Styler.call`, `styleFrom`, validation, semantics, focus, host independence, raw-spec precedence, enforced vertical order, and layout constraints.

Phase 5 - Fortal and application migration

- Add `fortalNavigationListStyle` and generated `FortalNavigationList`, reusing the ghost Toggle size2 recipe.
- Add a component worksheet and gallery example without treating the gallery as a second product consumer.
- Migrate the dashboard from its private widget to the public Fortal wrapper while leaving width, border, `SafeArea`, scrolling, brand, profile, drawer, breakpoints, routing, dismissal, and focus restoration in the application shell.

Completion gates

Completion requires every original promotion gate except the explicitly waived second-consumer gate. Package generation and exports must be deterministic; package and dashboard analysis and tests must pass; the public implementation must be no more conceptually complex than the private solution it replaces; and the final diff must contain no drawer, rail, route, compact-mode, or custom-row API.

For the maintenance gate, compare consumer-local duplicate production code and behavior ownership. Generated Mix surfaces, tests, and documentation are verification obligations rather than comparable consumer implementation LOC. The dashboard migration must shrink its private navigation source and remove its private data/widget concepts without creating a second interaction stack.

Decision rationale

The private spike proved the semantics and interaction boundary in both inline and drawer hosts. Publishing now is a deliberate bet that the generic destination/list contract is stable enough to be useful before a second product consumer exists. Risk is contained by keeping the API data-driven, vertical, controlled, host-agnostic, and built on existing `RemixToggle` behavior and `ToggleSpec` styling.

Amendment acceptance

- No drawer, rail, panel, sidebar, or routing API is introduced by Phases 4-5.
- Every public contract above is implemented, exported, documented, and directly tested.
- The Fortal recipe and dashboard migration preserve visual, pointer, keyboard, focus, semantics, wide-shell, and compact-drawer behavior.
- Any deviation discovered during implementation is recorded and must not widen the approved v1 scope.

RFC: Navigation Destinations and List Composition

Status: Dashboard-private spike approved; public API pending promotion gates

Repository context: conceptadev/remix

Scope: reusable navigation destination/list behavior; application-owned sidebar shell

Summary

Build and validate a dashboard-private navigation list that extracts the reusable destination contract from the current sidebar. The list owns controlled selection, destination interaction states, navigation semantics, section rendering, and style resolution. The application continues to own the complete sidebar shell: width, edge border, SafeArea, scrolling, brand/header, account/footer, responsive breakpoints, drawer or overlay presentation, routing, dismissal, and focus restoration.

Do not publish `RemixNavigationPanel` in this phase. The first implementation should reuse `RemixToggle` and its `NakedToggle` behavior rather than create a second focus, gesture, shortcut, feedback, and cursor stack. A public `RemixNavigationList<T>` is a candidate only after the private spike passes the promotion gates in this RFC.

Decision record

- Implementation authority: a dashboard-private spike is approved.
- Public API status: not approved yet; promotion depends on explicit gates.
- Candidate public root: `RemixNavigationList<T>`, not `RemixNavigationPanel<T>`.
- Public v1 anatomy, if promoted: sections and destinations only.
- Behavior owner: `RemixToggle` over `NakedToggle`; no custom Remix interaction stack.
- Semantics owner: an outer destination Semantics node with button and selected state; no toggled flag.
- Root semantics: a navigation landmark with explicit destination child nodes.
- Host ownership: width, border, SafeArea, scroll container, header, footer, breakpoint, drawer, routing, dismissal, and focus restoration remain application-owned.
- Compact icon-only mode: deferred from public v1 and may remain a dashboard experiment.
- Link destinations: outside this API; use `RemixLink` for true links and URL semantics.
- Portal styling: reuse the existing ghost `Toggle size2` recipe and token decisions.

- Radix parity: not applicable; any Fortal list recipe is a Flutter-specific extension.

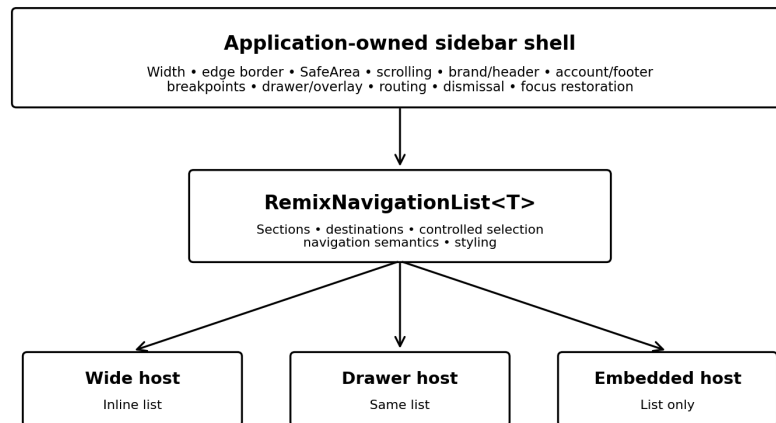


Figure 1. The reusable list sits inside an application-owned sidebar shell.

Problem and evidence

The current dashboard renders one Sidebar directly in the wide Row and also as the child of Material Drawer below its 720 logical-pixel breakpoint. This proves that the navigation body can be reused across hosts. It does not prove that the complete sidebar shell should become a public Remix component.

The existing Sidebar also owns dashboard-specific concerns: a fixed 256-pixel width, right-edge border, SafeArea, scrolling, brand header, dividers, profile/account menu, and spacing. Those concerns should not be folded into the reusable navigation contract.

Remix's current architecture makes interactive components visual layers over Naked UI.

RemixToggle already forwards `excludeSemantics` and exposes `selected`, `hovered`, `pressed`, `focused`, `focus-visible`, and `disabled` states through its `NakedToggle`-backed implementation. Reusing it keeps interaction behavior consistent with the rest of the package.

Resolved technical checks

- The repository pins Flutter `>= 3.41.0`, and Flutter 3.41 exposes `SemanticsRole.navigation`; the landmark contract is valid at the package floor.
- `NakedFocusableDetector` publishes focus semantics outside the visual child. Therefore the outer destination `Semantics` node must not set `excludeSemantics`; only `RemixToggle`

does so its toggled subtree is hidden while focusable/focused state can merge into the selected-button node.

- The outer `Semantics.onTap` is the assistive-technology action. Pointer and keyboard activation remain inside `NakedToggle`. Both routes call one `activate` callback and must each be tested for exactly one invocation.
- `RemixLink` already supplies true link semantics, optional `linkUrl` metadata, and Enter-only activation, so external links remain adjacent host composition rather than a navigation-destination variant.
- `semanticHint` exists on some whole-control APIs, but not on the closest item-data sibling. It is omitted from v1 destination data and can return only with a demonstrated consumer.

Scope

Dashboard-private spike includes

- An immutable section and destination data model.
- Controlled `selectedValue` and `onSelected` behavior.
- Destination activation for both unselected and already-selected destinations.
- Private destination data limited to value, required label, optional icon, and the one shared `Toggle` style used by the dashboard.
- Optional visual section labels.
- One navigation landmark and one selected-button semantic node per destination.
- Ordinary Tab and Shift+Tab traversal, with Enter and Space activation from `RemixToggle/NakedToggle`.
- One shared `Fortal Toggle` style applied to every destination; no public list `Styler`, `Spec`, or generated wrapper is required for the private spike.
- Current dashboard visual and behavior parity in the existing wide and drawer hosts.

Explicitly outside the spike and public v1

- A public `RemixNavigationPanel`, `RemixNavigationRail`, or `RemixNavigationDrawer` shell.
- Header or footer builders, brand content, account menus, or other shell chrome.
- Panel width, edge border, `SafeArea`, or scroll ownership.
- Compact icon-only public presentation, automatic tooltips, or a 64-pixel public width token.
- Routing, route parsing, URL synchronization, browser history, or back-button policy.
- Drawer or overlay lifecycle, scrim, gesture dismissal, Escape handling, or focus restoration.
- Nested navigation trees, submenus, expandable groups, badges, dividers, or arbitrary trailing widgets.
- Arrow-key roving focus, Home/End movement, or menu/tab semantics.
- A new public `NakedNavigationGroup` or `NakedNavigationDestination` API.

- Radix Themes parity claims.

Relationship to RemixToggleGroup

RemixToggleGroup<T> is the closest shipping sibling. Both are data-driven, controlled, single-select families with unique values, nullable selectedValue, item enabled/focus/autofocus/style inputs, raw styleSpec precedence, and values used as stable internal keys. The navigation list remains separate because its semantics and activation contract differ.

Earned divergences

- onSelected is ValueChanged<T>, never emits null, and fires on reselection because navigation activation can close a drawer, reset nested content, or scroll the current view. ToggleGroup's onChanged models a changed value instead of every activation.
- The navigation list uses ordinary Tab traversal and a navigation landmark. ToggleGroup uses roving arrow-key focus and toggle-option semantics.
- T extends Object reserves null as the no-selection sentinel at compile time. This is a candidate consistency improvement to back-port to RemixToggleGroup, not a reason to weaken this API.
- Destination visuals reuse ToggleSpec because each destination renders a RemixToggle. ToggleGroupItemSpec remains correct for ToggleGroup because its options are not RemixToggle widgets.
- Enabled, semanticLabel, focusNode, autofocus, and a per-item ToggleStyler remain candidate fields because the closest sibling already exposes the same composability.

Deliberately omitted from v1

- Item keys are not public; required-unique values provide internal ValueKey identities.
- Icons are optional because a required label supplies the visible and accessible name.
- Selected icons, semantic hints, per-item cursor, and per-item feedback are omitted until a real consumer demonstrates them.
- Section and destination lists follow the existing documented immutability contract; the widget does not copy them on every build.

Candidate public API

The following API is a candidate for promotion after the private spike. It is not approved merely because it appears in this RFC. The API stays narrow and reuses ToggleStyler for destination-level visual overrides.

```
class RemixNavigationSection<T extends Object> {
```



```

    const RemixNavigationSection({
      this.label,
      required this.destinations,
    });

    final String? label;
    final List<RemixNavigationDestination<T>> destinations;
  }

class RemixNavigationDestination<T extends Object> {
  const RemixNavigationDestination({
    required this.value,
    required this.label,
    this.icon,
    this.semanticLabel,
    this.enabled = true,
    this.focusNode,
    this.autofocus = false,
    this.style = const ToggleStyler.create(),
  });

  final T value;
  final String label;
  final IconData? icon;
  final String? semanticLabel;
  final bool enabled;
  final FocusNode? focusNode;
  final bool autofocus;
  final ToggleStyler style;
}

class RemixNavigationList<T extends Object> extends StatelessWidget {
  const RemixNavigationList({
    super.key,
    required this.sections,
    required this.selectedValue,
    this.onSelected,
    this.enabled = true,
    this.semanticLabel,
    this.excludeSemantics = false,
    this.style = const NavigationListStyler.create(),
    this.styleSpec,
  });

  final List<RemixNavigationSection<T>> sections;
  final T? selectedValue;
  final ValueChanged<T>? onSelected;
  final bool enabled;
  final String? semanticLabel;
  final bool excludeSemantics;
  final NavigationListStyler style;
  final NavigationListSpec? styleSpec;

  static final styleFrom = NavigationListStyler.new;
}

```

Selection and validation contract

- `selectedValue` is the sole source of selected state; null means no destination is selected.
- In debug builds, a non-null `selectedValue` must match exactly one destination across all sections. Validation follows the sibling's assert-based precedent; in release builds an unmatched value renders every destination unselected rather than throwing.
- Destination values must be unique across the full list and keep stable equality and hash-code behavior.
- `onSelected` fires once for every enabled activation, including reselection of the current destination.
- A null `onSelected` or `enabled == false` disables every destination while preserving visual `selectedValue`. The list also publishes `WidgetState.disabled` to its root style context so container and default-destination variants resolve consistently.
- Per-destination enabled state combines with root enabled state.
- Destination labels are required and nonblank. Semantic labels and section labels must not be blank when supplied; icons are optional.
- At most one destination may request autofocus.
- Do not mutate section or destination lists after constructing the widget; rebuild with new lists when content or order changes, matching `RemixToggleGroup`'s public contract. An empty sections list is valid when `selectedValue` is null; empty sections are skipped.

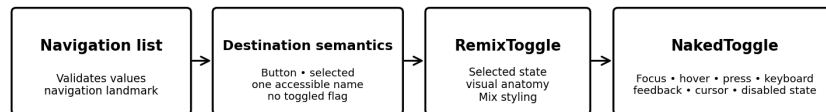
Behavior and accessibility

Each destination should reuse `RemixToggle` for visual state and `NakedToggle` behavior. The normal `Toggle` semantics are excluded, then replaced by one outer semantic node that expresses navigation selection rather than binary on/off state.

```
Semantics(  
  button: true,  
  selected: selected,  
  enabled: effectiveEnabled,  
  label: destination.semanticLabel ?? destination.label,  
  onTap: effectiveEnabled ? activate : null,  
  child: RemixToggle(  
    selected: selected,  
    enabled: effectiveEnabled,  
    excludeSemantics: true,  
    onChanged: (_) => activate(),  
    label: destination.label,  
    icon: destination.icon,  
    focusNode: destination.focusNode,  
    autofocus: destination.autofocus,  
    style: resolvedStyle,  
  ),  
)
```

The outer `onTap` is the assistive-technology activation path. Pointer and keyboard activation remain inside `NakedToggle`. Both paths call the same `activate` callback but serve disjoint input routes; tests assert exactly one callback per route.

Do not set `excludeSemantics` on the outer node. `NakedFocusableDetector` publishes focusable/focused semantics outside `RemixToggle`'s excluded visual subtree, and the outer selected-button node must allow those focus properties to merge.



Outer selected-button semantics replace `Toggle`'s normal toggled semantics.
Naked UI remains the interaction and focus owner.

Figure 2. Navigation semantics replace `Toggle`'s toggled semantics while Naked UI retains interaction ownership.

Root semantics

- Wrap the rendered list in `Semantics(role: SemanticsRole.navigation, container: true, explicitChildNodes: true)`. The repository's Flutter floor is 3.41, where `SemanticsRole.navigation` is available.
- Use `semanticLabel` to distinguish multiple navigation landmarks when needed.
- Each non-null section label is a separate semantic header node and must not merge into destination accessible names. Empty sections are skipped with their labels so dynamic lists do not render orphan headings.
- `excludeSemantics` hides the complete navigation landmark and its destination subtree.

Keyboard and focus

- `Tab` and `Shift+Tab` use ordinary widget traversal in visual order.
- `Enter` and `Space` activate through `NakedToggle`'s button shortcuts.
- No arrow-key roving focus in v1.
- Focus-visible state resolves through the existing `Remix/Naked/Mix` path.
- Caller-owned focus nodes remain valid when the host rebuilds or moves the list between inline and drawer composition.
- Escape handling and focus restoration remain the drawer or overlay host's responsibility.
- Directional alignment and ordering use ambient text direction.

Button navigation versus links

This contract targets in-app view selection. Destinations publish button semantics and support Space because activation changes the active view in place. A destination that represents a true document or URL link should use `RemixLink` so it can publish link semantics, optional `linkUrl` metadata, and Enter-only keyboard activation. Hosts compose `RemixLink` rows adjacent to the navigation list; links are intentionally not encoded as navigation-destination data.

Style and spec anatomy

The list should add one new `Spec/Styler` family and reuse `ToggleSpec` for destination visuals. This avoids duplicating container, label, icon, selected-state, focus, and disabled styling that already exists in `RemixToggle`. This reuse is deliberate: navigation destinations are rendered as `RemixToggle` instances, while `RemixToggleGroup` items have their own `ToggleGroupItemSpec` because they are not `RemixToggle` widgets.

```
@MixableSpec(  
  target: RemixNavigationList.new,  
  extraStylerMixins: [RemixBoxStylerMixin],  
)  
class NavigationListSpec with _$NavigationListSpec {  
  @override  
  @MixableField(forwardStyler: true)  
  final StyleSpec<FlexBoxSpec> container;  
  
  @override  
  final StyleSpec<FlexBoxSpec> section;  
  
  @override  
  final StyleSpec<TextSpec> sectionLabel;  
  
  @override  
  final StyleSpec<FlexBoxSpec> destinations;  
  
  @override  
  final StyleSpec<ToggleSpec> destination;  
}
```

Styler rules

- `container` is the forwarded vertical `FlexBox` surface for the section list; its flex spacing controls space between sections. It does not imply width, border, `SafeArea`, or scrolling. Root style resolution receives `WidgetState.disabled` when `!enabled || onSelected == null`.
- `section` controls the vertical layout of one optional label plus its destination group; its flex spacing controls the label-to-group gap.
- `sectionLabel` controls visual section-label typography only.
- `destinations` is a vertical `FlexBox` whose spacing controls the gap between destination rows.

- The widget enforces vertical direction after style resolution for container, section, and destinations so fluent styles and raw specs cannot reorder the visual or semantic navigation sequence.
- destination holds the default ToggleSpec for every destination.
- Each destination's ToggleStyler merges after the list's default destination style.
- When styleSpec is supplied, it is authoritative and bypasses root and per-destination fluent styles.
- The list does not add a selected-indicator slot in v1; add one only after a real design requires anatomy beyond ToggleSpec.

Fortal recipe and shell boundary

Fortal should style the list through the existing ghost Toggle size2 recipe. The application shell keeps the current dashboard width, panel fill, edge border, SafeArea, dividers, scrolling, brand, profile menu, and responsive host placement.

```
@MixWidget(target: RemixNavigationList.new)
NavigationListStyler fortalNavigationListStyle({
  bool highContrast = false,
}) {
  return NavigationListStyler().destination(
    fortalToggleStyle(
      variant: .ghost,
      size: .size2,
      highContrast: highContrast,
    ).container(
      .mainAxisSize(.max)
      .mainAxisAlignment(.start),
    ),
  );
}
```

Fortal decisions

- Idle label/icon: gray12 with medium text.
- Geometry baseline: existing Toggle size2 padding, gap, radius, text2, and icon space4.
- Hover: grayA3; press: grayA4.
- Selected: accent3 background with accent11 content; accent12 content in high contrast.
- Selected hover: accent4; selected press: accent5.
- Focus visible: existing Fortal focus-ring helper.
- Disabled: grayA3 background with gray8 label and icon.
- Section-label spacing and typography are recorded in a component worksheet and should prefer existing Fortal tokens before adding new tokens.
- No panel width or compact width token is part of the reusable list API.
- The component remains outside the Radix parity manifest.

Application composition

The dashboard keeps its existing shell and places the same navigation list inside that shell for both wide and drawer hosts.

```
Widget buildDashboardSidebar() {  
  return Container(  
    width: 256,  
    decoration: dashboardSidebarDecoration,  
    child: SafeArea(  
      child: Column(  
        children: [  
          const DashboardBrand(),  
          const FortalDivider(size: .size4),  
          Expanded(  
            child: SingleChildScrollView(  
              child: FortalNavigationList<DashboardPage>(  
                sections: sections,  
                selectedValue: selected,  
                onSelected: selectPage,  
              ),  
            ),  
          ),  
          const FortalDivider(size: .size4),  
          const DashboardAccountMenu(),  
        ],  
      ),  
    ),  
  );  
}
```

The wide Row renders buildDashboardSidebar() inline. The compact Scaffold.drawer renders the same shell. Drawer close behavior remains in DashboardShell's selection callback.

Public promotion gates

- Boundary gate: the implementation is a list/destination primitive and contains no application-shell concerns.
- Behavior gate: interaction is provided by RemixToggle/NakedToggle; no duplicate Flutter behavior stack exists.
- Evidence gate: at least one second real application consumer exists before any public API is approved. A gallery example alone is not a second consumer.
- Semantics gate: navigation landmark, section-header semantics, selected destination semantics, disabled actions, empty-section behavior, and duplicate-name tests pass.
- Host gate: ordinary use requires no Material ancestor, Overlay, Navigator, SafeArea, or router.
- Platform gate: documentation clearly distinguishes in-app button navigation from RemixLink.
- Style gate: ToggleSpec and fortalToggleStyle are reused rather than copied into a parallel destination system.

- API gate: package-barrel, styleFrom, generic Styler.call<T>, raw-spec precedence, and generated-wrapper tests pass.
- Maintenance gate: the public implementation reduces net code and concepts compared with the dashboard-private solution.
- Compact gate: icon-only mode remains private until tooltip, host capability, width, high-contrast, and accessibility behavior are proven.

Implementation phases

0. Correct the current dashboard assumption

Update the stale Sidebar comment that says RemixToggle cannot opt out of semantics. It now supports excludeSemantics. Preserve the current selected-without-toggled regression test.

1. Dashboard-private list spike

Extract only the data the dashboard consumes: value, required label, optional icon, and one shared Toggle style. Replace the private section loop and destination widget with a private list that uses RemixToggle plus outer selected semantics. Keep disabled-item controls, caller focus nodes, autofocus, semantic-label overrides, and per-item styles as promotion-time decisions unless a real consumer requires them. Keep all shell chrome outside the list.

2. Measure the result

Compare code size, concept count, visual parity, semantics, focus behavior, and test complexity against the original Sidebar. Record whether the abstraction is genuinely simpler.

3. Decide public promotion

Evaluate every promotion gate. If a gate fails, keep the implementation dashboard-private and document the reason. Do not create package exports or compatibility commitments by default.

4. Public vertical slice, only if approved

Add RemixNavigationList, section and destination data types, NavigationListSpec/Styler, exports, documentation, public API tests, generic codegen coverage, and host compatibility tests.

5. Fortal recipe and migration

Add a generated FortalNavigationList<T> wrapper, component worksheet, and gallery example. Migrate the dashboard only after all existing dashboard tests remain green and the public list is simpler than the private version.

Verification plan

Private spike tests

- Wide and drawer hosts render the same destination data and current 256-pixel shell.
- Activating unselected and selected destinations calls onSelected exactly once.
- Drawer selection still closes through the host callback.
- Current visual treatment matches the dashboard's Fortal ghost Toggle baseline. High-contrast mode keeps selected state visually distinguishable; if the baseline fails, change the Fortal recipe rather than the base list anatomy.
- Exactly one named semantic node exists per destination, and the node reports focusable/focused state to assistive technology.
- Invoking the destination's semantic tap action calls onSelected exactly once.
- Selected destinations expose selected == true and no toggled flag.
- Root list exposes SemanticsRole.navigation; non-null section labels are separate semantic headers, and empty sections do not render orphan labels.
- Tab order, Enter, Space, disabled state, focus-visible state, and RTL remain correct.
- Long labels and 200% text scaling do not cause render overflow in the existing expanded shell.
- The list builds inside a SingleChildScrollView with unbounded height.
- The list builds inside a fixed-height Column without taking over scrolling.
- Brand, account menu, SafeArea, width, border, scrolling, and dividers remain outside the list.

Public API tests, only after promotion

- Construct list, section, destination, and NavigationListSpec from the package barrel.
- RemixNavigationList.styleFrom returns NavigationListStyler.
- NavigationListStyler.call<T> preserves generic T and forwards the owning style.
- Raw NavigationListSpec bypasses root and per-destination fluent styles.
- Fluent and raw flex directions cannot change the enforced vertical navigation order.
- FortalNavigationList<T> generation forwards sections, selection, callbacks, semantics, enabled state, and type T.
- Duplicate values, unmatched selected values, blank labels, and multiple autofocus requests fail clearly in debug builds; release mode renders an unmatched selected value as no selection. Empty lists and empty sections follow the documented behavior.

- The list builds under WidgetsApp with no Material ancestor, Overlay, or Navigator.
- True link examples use RemixLink instead of navigation destinations.

Risks and rejected alternatives

Public panel shell

Rejected for now because it would own shell-specific width, edge, SafeArea, scrolling, header, footer, and placement assumptions from one application.

Custom Remix behavior layer

Rejected because it would duplicate NakedToggle's focus, shortcuts, feedback, cursor, press, hover, and disabled behavior, contrary to the current Remix architecture.

Compact public mode

Deferred because icon-only discoverability normally needs tooltip help, which introduces an Overlay capability question that ordinary list usage does not otherwise have.

NakedToggleGroup

Rejected because it introduces toggle-option semantics and roving arrow-key focus that are not part of this v1 navigation contract.

RemixLink for all destinations

Rejected because the dashboard destinations select in-app views and support button-style Space activation. True links remain a separate RemixLink use case.

Acceptance criteria for the approved spike

- The dashboard has one immutable section/destination model used in both wide and drawer hosts.
- The private destination model contains only value, required label, optional icon, and one shared Toggle style; promotion-only fields are not implemented prematurely.
- The reusable unit is a list/destination implementation; no public panel, rail, or drawer shell is added.
- Each destination reuses RemixToggle/NakedToggle behavior and has one outer selected semantic node.

- The outer node preserves focusable/focused semantics, and its semantic tap action invokes onSelected exactly once.
- The list publishes a navigation landmark, non-null section labels publish separate header semantics, empty sections are skipped, and no destination publishes toggled semantics.
- Host-owned width, border, SafeArea, scrolling, header, footer, routing, drawer lifecycle, dismissal, and focus restoration remain outside the list.
- Current dashboard visuals and all existing dashboard behavior and semantics tests remain green.
- The spike records whether the abstraction reduces net code and concepts.
- Public exports, compatibility commitments, compact mode, and the Fortal wrapper are added only after a second real application consumer exists and all relevant promotion gates pass.

References

[Dashboard shell](#)

[Dashboard sidebar](#)

[Dashboard smoke tests](#)

[RemixToggle implementation](#)

[ToggleSpec](#)

[RemixLink implementation](#)

[Host capability tests](#)

[Host ownership review, issue #82](#)

[Host capability contract, issue #96](#)

[Destination semantics context, issue #119](#)

[Fortal Toggle recipe](#)

[RemixToggleGroup implementation](#)

[NakedFocusableDetector](#)

[Remix package SDK floor](#)

[Flutter 3.41 semantics roles](#)